ConsistentRFT: Reducing Visual Hallucinations in Flow-based Reinforcement Fine-Tuning

Under Review

Xiaofeng Tan Jun Liu Yuanting Fan Bin-Bin Gao Xi Jiang Xiaochen Chen Jinlong Peng Chengjie Wang Hongsong Wang Feng Zheng

Southeast University · Southern University of Science and Technology · Tencent Youtu Lab

Paper PDF | Project Page | Model HuggingFace

TL;DR: We analyze why visual hallucinations arise in reinforcement fine-tuning from two perspectives — **limited exploration** and **trajectory imitation** — and propose **Dynamic Granularity Rollout (DGR)** and **Consistent Policy Gradient Optimization (CPGO)** to address them.

📑 Table of Contents

- 📣 News
- 📅 Plan
- 🧬 Model Zoo
- 📦 Preprocess Data
- ⚙️ Setup
- 🚀 Training
- 🎨 Inference
- 📊 Evaluation
- 🙏 Acknowledgement
- 📝 Citation

📣 News

- **[2024/12]** The paper has been publicly released.

📅 Plan

- [x] Release paper
- [x] Release checkpoints
- [x] Release evaluation code:
  - [ ] In-domain / out-of-domain evaluation
  - [x] High-level visual hallucination evaluation (v1 & v2)
  - [x] Low-level visual hallucination evaluation
- [ ] Release training code:
  - [ ] Dynamic Granularity Rollout (DGR)

    - ☑ Consistent Policy Gradient Optimization (CPGO)
    - ☑ ConsistentRFT for DPO, GRPO, and DDPO
- ☑ Release inference code

📦 Preprocess Data

Adjust the `prompt_path` parameter in `./scripts/preprocess/preprocess_flux_rl_embeddings.sh` to obtain the embeddings of the prompt dataset:

```
bash scripts/preprocess/preprocess_flux_rl_embeddings.sh
```

⚙️ Setup

> ⚠ **Important:** The training and evaluation environments have **different dependency versions** (e.g., PyTorch, vLLM, flash-attn). You **must** use separate conda environments to avoid conflicts.

1. Training Environment (`ConsistentRFT`)

```
cd ConsistentRFT
conda create -n ConsistentRFT python=3.12
conda activate ConsistentRFT

# Default: PyTorch with CUDA 12.8. For CUDA 12.4, set TORCH_CUDA=cu124
bash env_setup.sh
# or: TORCH_CUDA=cu124 bash env_setup.sh
```

> **Note:** `env_setup.sh` will automatically install PyTorch, Flash Attention 2 (pre-built wheel from GitHub Releases), HPSv2, open_clip, and all training dependencies. The script auto-detects your torch version, Python version, and CXX11 ABI to download the correct flash-attn wheel. CUDA 12.x is required for Hopper GPUs (H20/H100/H200).

2. Reward Evaluation Environment (`ConsistentRFT_Eval`)

The reward evaluation (HPS-v2.1, PickScore, ImageReward, CLIP Score, Aesthetic Predictor, UnifiedReward) uses a **separate** conda environment to avoid dependency conflicts with training.

```
conda create -n ConsistentRFT_Eval python=3.12
conda activate ConsistentRFT_Eval
bash env_eval_setup.sh
```

> **Note:** `env_eval_setup.sh` will install PyTorch, vLLM, flash-attn, and all reward model dependencies (image-reward, clip, aesthetic-predictor-v2-5, open_clip, HPSv2, etc.).

2.1 HPS-v2.1

Clone the HPSv2 repository:

```
git clone https://github.com/tgxs002/HPSv2.git
```

Download the model weights to `./pretrained_ckpt`:

```
huggingface-cli login
huggingface-cli download --resume-download xswu/HPSv2
HPS_v2.1_compressed.pt --local-dir ./pretrained_ckpt/
huggingface-cli download --resume-download laion/CLIP-ViT-H-14-laion2B-
s32B-b79K open_clip_pytorch_model.bin --local-dir ./pretrained_ckpt/
```

2.2 PickScore

Refer to PickScore for more details.

Download the required models:

```
huggingface-cli download --resume-download yuvalkirstain/PickScore_v1
huggingface-cli download --resume-download laion/CLIP-ViT-H-14-laion2B-
s32B-b79K
```

2.3 ImageReward

Refer to ImageReward for more details.

Download the model weights to `./pretrained_ckpt/imagerward`:

```
mkdir -p pretrained_ckpt/imagerward
huggingface-cli login
huggingface-cli download --resume-download THUDM/ImageReward
med_config.json --local-dir ./pretrained_ckpt/imagerward/
huggingface-cli download --resume-download THUDM/ImageReward
ImageReward.pt --local-dir ./pretrained_ckpt/imagerward/
```

2.4 CLIP Score

Refer to CLIP for more details.

Download the required model:

```
huggingface-cli download --resume-download apple/DFN5B-CLIP-ViT-H-14-384
```

2.5 Aesthetic Predictor V2.5

Refer to improved-aesthetic-predictor for more details.

```
conda activate ConsistentRFT_Eval
pip install aesthetic-predictor-v2-5
```

2.6 UnifiedReward

Refer to UnifiedReward for more details.

UnifiedReward is a VLM-based reward model that requires deploying a vLLM server. It supports two evaluation modes: **semantic** (Alignment/Coherence/Style scores) and **score** (Final Score 1-5).

Serve the model (using the `ConsistentRFT_Eval` environment):

For the 7B model (1 GPU):

```
conda activate ConsistentRFT_Eval
CUDA_VISIBLE_DEVICES=0 vllm serve ./pretrained_ckpt/unified_reward \
    --served-model-name UnifiedReward \
    --trust-remote-code \
    --gpu-memory-utilization 0.9 \
    --tensor-parallel-size 1 \
    --limit-mm-per-prompt '{"image": 2}' \
    --port 8080
```

For the 72B model (4 GPUs):

```
conda activate ConsistentRFT_Eval
CUDA_VISIBLE_DEVICES=0,1,2,3 vllm serve CodeGoat24/UnifiedReward-2.0-
qwen-72b \
    --served-model-name UnifiedReward \
    --trust-remote-code \
    --tensor-parallel-size 4 \
    --limit-mm-per-prompt '{"image": 16}' \
    --port 8080
```

Available model variants:

Model	HuggingFace ID	Size
UnifiedReward 2.0 Qwen 3B	`CodeGoat24/UnifiedReward-2.0-qwen-3b`	3B
UnifiedReward 2.0 Qwen 7B	`CodeGoat24/UnifiedReward-2.0-qwen-7b`	7B
UnifiedReward 2.0 Qwen 32B	`CodeGoat24/UnifiedReward-2.0-qwen-32b`	32B
UnifiedReward 2.0 Qwen 72B	`CodeGoat24/UnifiedReward-2.0-qwen-72b`	72B

> **Note:** The server must be running at `http://127.0.0.1:8080` (default in `eval_reward.sh`) before running evaluation with `unified_reward` or `all` reward model type. If the server URL differs, modify `unified_reward_url` in `scripts/evaluate/eval_reward.sh`.

3. VH Evaluator Environment (`VH_Eval`)

The Visual Hallucinations Evaluator requires a **different vLLM version**. Create a third conda environment:

```
conda create -n VH_Eval python=3.12
conda activate VH_Eval
pip install vllm==0.9.0.1 transformers==4.52.4
```

🚀 Training

ConsistentRFT GRPO Fine-tuning (8 GPUs)

```
bash scripts/finetune/FLUX_GRPO.sh
```

Online DPO Fine-tuning (8 GPUs)

```
bash scripts/finetune/FLUX_DPO.sh
```

🎨 Inference

We provide inference scripts for both **LoRA fine-tuned** and **full fine-tuned** Flux models.

1. LoRA Fine-tuned Models

```
bash scripts/inference/multi_inference_flux_lora.sh
<checkpoint_base_dir> <gpu_ids> <num_gpus>
```

Example:

```
bash scripts/inference/multi_inference_flux_lora.sh
./data/outputs/grpo_consist_mix_lora 0,1,2,3 4
```

2. Full Fine-tuned Models

```
bash scripts/inference/multi_inference_flux.sh <checkpoint_base_dir>
<gpu_ids> <num_gpus>
```

Example:

```
bash scripts/inference/multi_inference_flux.sh
./data/outputs/consistentrft_full 0,1,2,3 4
```

3. Arguments

Argument	Description	Example
`checkpoint_base_dir`	Base directory containing multiple `*checkpoint-*` subdirectories	`./data/outputs/grpo_consist_mix_lora`
`gpu_ids`	CUDA visible devices	`0,1,2,3`
`num_gpus`	Number of GPUs per node	`4`

> **Note:** The script automatically discovers all `*checkpoint-*` subdirectories under `checkpoint_base_dir` and runs inference for each.

📊 Evaluation

1. Reward Model Evaluation

Evaluate generated images using multiple reward models (HPS-v2.1, CLIP Score, ImageReward, PickScore, etc.):

```
bash scripts/evaluate/eval_reward.sh <checkpoint_dir> <gpu_ids>
<num_gpus>
```

Example:

```
bash scripts/evaluate/eval_reward.sh ./outputs/checkpoint-100 0,1,2,3 4
```

Argument	Description
`checkpoint_dir`	Directory containing the checkpoint with generated samples
`gpu_ids`	CUDA visible devices
`num_gpus`	Number of GPUs per node

> **Note:** Modify `reward_model_type` in the script to select specific reward models (`"hpsv2"`, `"clip_score"`, `"image_reward"`, `"pick_score"`, `"unified_reward"`, or `"all"`).

2. High-Level Visual Hallucinations Evaluator

This evaluator uses a Vision-Language Model (VLM) to detect over-optimization hallucinations in generated images, including **grid pattern artifacts** and **texture over-enhancement**.

We provide two versions:

- **VH_Evaluator**: Standard evaluation using original images
- **VH_Evaluator_v2**: Enhanced evaluation with image preprocessing for improved grid pattern detection

2.1 Prerequisites

- Download Qwen2.5-VL-72B-Instruct to `./pretrained_ckpt/Qwen2.5-VL-72B-Instruct`

2.2 Start Server

Launch the VLM server (default: 8 GPUs, port 8000):

```
cd ./VH_Evaluator
bash start_server.sh [NUM_GPUS] [PORT]
```

Examples:

```
bash start_server.sh          # 8 GPUs, port 8000
bash start_server.sh 4        # 4 GPUs, port 8000
bash start_server.sh 4 8001   # 4 GPUs, port 8001
```

2.3 Standard Evaluation (VH_Evaluator)

Run the evaluation client to analyze images in a folder:

```
cd ./VH_Evaluator
bash eval.sh <image_folder> <output_json_path>
```

Example:

```
bash eval.sh ./outputs/checkpoint-100/samples
High_Level_eval/checkpoint-100.json
```

2.4 Enhanced Grid Detection (VH_Evaluator_v2)

> 🚧 **TODO:** This section is still being organized. Details coming soon.

```
cd ./VH_Evaluator_v2
bash eval.sh <image_folder> <output_json_path>
```

Example:

```
bash eval.sh ./outputs/checkpoint-100/samples
High_Level_eval/checkpoint-100_v2.json
```

> **Note:** VH_Evaluator_v2 uses more API tokens due to multiple preprocessed images per evaluation. Use it when grid pattern detection accuracy is critical.

Argument	Description
`image_folder`	Path to the folder containing generated images
`output_json_path`	Path to save the evaluation results (JSONL format)

Output Format

The output is a JSONL file (one JSON object per line). Each line contains the evaluation result for a single image:

```
{
  "image_name": "10.jpg",
  "image_path": "/path/to/sample_folder/10.jpg",
  "evaluation": {
    "image": {
      "has_over_optimization": false,
```

```
        "over_optimization_level": 1,
        "detail_sharpness_score": 1,
        "irrelevant_details_score": 0,
        "grid_pattern_score": 0,
        "overall_hallucination_score": 0.67,
        "confidence": 0.9,
        "detailed_analysis": "The image appears to have minimal over-
optimization..."
      }
    },
    "attempt": 1,
    "success": true,
    "timestamp": "2026-03-05T01:43:17.815411"
  }
```


Field	Type	Description
`has_over_optimization`	bool	Whether over-optimization hallucination is detected
`over_optimization_level`	int (0-5)	Overall severity of over-optimization
`detail_sharpness_score`	int (0-5)	Degree of unnaturally sharp edges / over-refined textures
`irrelevant_details_score`	int (0-5)	Degree of irrelevant / excessive decorative details
`grid_pattern_score`	int (0-5)	Degree of grid pattern artifacts
`overall_hallucination_score`	float (0-5)	Weighted overall hallucination score
`confidence`	float (0-1)	Model's confidence in the assessment
`detailed_analysis`	string	Free-text explanation of the evaluation

3. Low-Level Visual Hallucinations Evaluator

This evaluator computes image quality metrics to detect low-level visual artifacts, including **sharpness**, **high-frequency energy**, **edge artifacts**, and **noise level**.

3.1 Evaluation

```
python VH_Evaluator/low_level_metric.py <input_dir> <output_dir> [--
verbose]
```

Example:

```
python VH_Evaluator/low_level_metric.py ./outputs/checkpoint-100/samples
./Low_Level_eval --verbose
```

Argument	Description
`input_dir`	Path to the folder containing generated images
`output_dir`	Directory to save the evaluation results (JSON format)
`--verbose`	(Optional) Show detailed per-image analysis information

🙏 Acknowledgement

This work is built on many amazing research works and open-source projects. Thanks to all the authors for sharing!

- DanceGRPO
- Flow-GRPO
- MixGRPO
- DDPO
- Pref-GRPO

📝 Citation

If you find this repository helpful in your research, please consider citing the paper and starring the repo.

```
@article{tan2026consistentrft,
  title={ConsistentRFT: Reducing Visual Hallucinations in Flow-based
Reinforcement Fine-Tuning},
  author={Tan, Xiaofeng and Liu, Jun and Fan, Yuanting and Gao, Bin-Bin
and Jiang, Xi and Chen, Xiaochen and Peng, Jinlong and Wang, Chengjie
and Wang, Hongsong and Zheng, Feng},
  journal={arXiv preprint arXiv:2602.03425},
  year={2026}
}
```